TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO THỰC HÀNH HỆ ĐIỀU HÀNH

Lab 02

Thiết kế và kiểm chứng lời gọi hệ thống trên xv6-riscv

System Calls: Trace, Sysinfo, Process Tree

Lớp

CQ2024/22

Giảng viên hướng dẫn

ThS. Lê Viết Long

ThS. Phạm Tuấn Sơn

Nhóm thực hiện

Họ và tên	Mã số sinh viên
Phan Lê Đăng Khoa	24120347
Âu Bảo Trân	24120468
Dương Nhựt Duy	24120293

Thủ Đức, TP. Hồ Chí Minh – tháng 11 năm 2026

Mục lục

1 Giới thiệu và môi trường **1**

1.1 Mục tiêu và phạm vi 1

1.2 Môi trường và bản mã nguồn 1

2 Thiết kế và triển khai **2**

2.1 Đường đi của một lời gọi hệ thống 2

2.2 Trace: trạng thái riêng và kế thừa 2

2.3 Sysinfo: thống kê tài nguyên kernel 3

2.4 Ptree và pstree: tách thu thập khỏi hiển thị 4

3 Kiểm thử và đánh giá **5**

3.1 Khả năng tái lập và tiêu chí 5

3.2 Ma trận kiểm thử trên bản tích hợp 5

3.3 Minh chứng chạy thật và phân tích 6

3.4 Lỗi được phát hiện trong rà soát tích hợp 6

4 Khó khăn và giới hạn **7**

5 Phân công và đóng góp **7**

6 Kết luận **8**

Tài liệu tham khảo **8**

Thiết kế và kiểm chứng lời gọi hệ thống trên xv6-riscv

Phan Lê Đăng Khoa Âu Bảo Trân Dương Nhựt Duy

Tóm tắt. Báo cáo trình bày ba lời gọi hệ thống bổ sung cho xv6-riscv: theo dõi lời gọi theo bitmask, thu thập bộ nhớ trống và số tiến trình, lấy thông tin để dựng cây tiến trình trong không gian người dùng. Mã nguồn kế thừa Lab 1 và được tích hợp từ ba nhánh có lịch sử riêng. Rà soát phát hiện lỗi kiểm tra giới hạn bộ đệm và byte chưa khởi tạo trong bản ghi process tree; các lỗi được tái hiện trước khi sửa. Thực nghiệm trên QEMU gồm bảy nhóm kiểm thử trace, chín nhóm kiểm thử tích hợp mới, các bộ hồi quy Lab 1 và bộ `usertests -q` của xv6. Kết quả kiểm chứng chức năng được ghi theo commit và nhật ký thực thi; báo cáo phân biệt kết quả quan sát với giới hạn về snapshot đồng thời và hiệu năng.

Từ khóa: xv6-riscv; syscall; bitmask; copyout; khóa; cây tiến trình.

1 Giới thiệu và môi trường

1.1 Mục tiêu và phạm vi

Lab 2 yêu cầu thêm `trace`, `sysinfo` và `ptree`, đồng thời viết các chương trình `trace`, `sysinfotest` và `pstree`. Đề `Lab2_OS_System_Call.docx` là căn cứ về chức năng; hướng dẫn MIT năm 2020 được dùng để đối chiếu trace và sysinfo [1, 2]. Process tree là phần mở rộng trong đề của môn học.

Ba câu hỏi đánh giá gồm: **RQ1**, trạng thái theo dõi có được giữ đúng qua `fork`/`exec` và cách ly giữa các tiến trình không; **RQ2**, dữ liệu kernel có được thu thập dưới khóa phù hợp và chuyển về đúng phạm vi user memory không; **RQ3**, ba phần có hoạt động cùng nhau và giữ được các chức năng sẵn có không. Báo cáo kiểm chứng tính đúng và xử lý lỗi, không đo hiệu năng hay suy ra điểm môn học.

1.2 Môi trường và bản mã nguồn

Bảng 1. Cấu hình kiểm thử được ghi nhận ngày 04/11/2026.

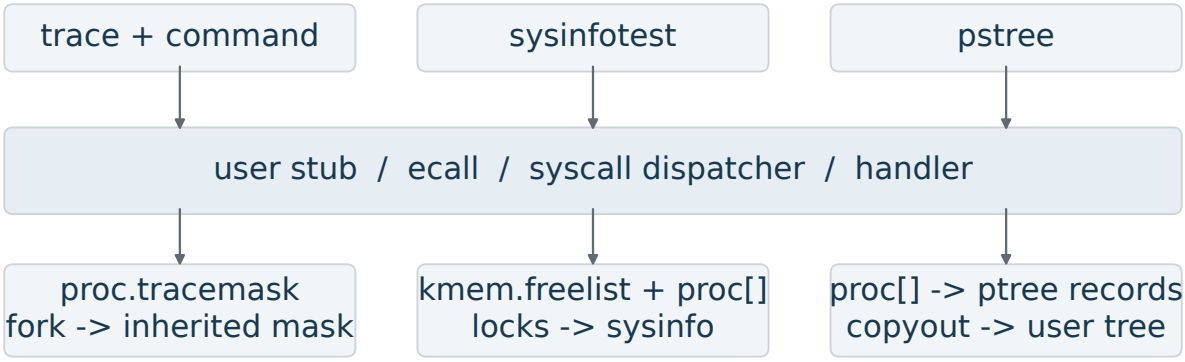
Thành phần	Cấu hình
Máy chủ	Ubuntu 24.04.5 LTS, WSL2/Linux 6.6.87.2, x86_64; AMD Ryzen 5 5600H.
Công cụ	RISC-V GCC 13.3.0; QEMU 8.2.2; Python 3.12; GDB đa kiến trúc.
Máy giả lập	RISC-V `virt`, 128 MiB RAM, 3 CPU ảo; virtio-blk.
Baseline nhóm	Lab 1: `201dd84`; chuẩn bị chung: `fcbb90e`.
Bản kiểm chứng	Commit `626c6e0`; đầy đủ SHA và lệnh chạy trong `docs/lab02/validation/run-context.json`.
Giới hạn	`NPROC=64`, `PGSIZE=4096`, `USERSTACK=2`; cấu hình `LAB=util` của scaffold được giữ lại.

Giữ `LAB=util` bảo toàn cấu hình Lab 1. Chế độ `LAB=syscall` trong scaffold MIT 2024 trỏ đến bài `attack`, khác đề hiện tại; tên cấu hình không phải số thứ tự lab của môn học. Chỉ các lệnh test được nêu trong báo cáo mới được dùng làm căn cứ kết quả.

2 Thiết kế và triển khai

2.1 Đường đi của một lời gọi hệ thống

Giao diện user được khai báo trong `user/user.h`; `user/usys.pl` sinh stub đưa số hiệu vào thanh ghi `a7` rồi thực hiện `ecall`. Sau khi trap vào kernel, dispatcher trong `kernel/syscall.c` chọn handler. Handler lấy tham số bằng `argint`/`argaddr`, xử lý và trả kết quả qua `a0`. Cách tổ chức được giữ theo xv6 hiện có [3, 4].



Hình 1. Phân tách xử lý user/kernel của ba syscall; đây là sơ đồ thiết kế, không phải số liệu đo.

Ba số hiệu được chốt trước khi chia nhánh: `SYS_trace=22` cho trace, `SYS_sysinfo=23` cho sysinfo và `SYS_ptree=24` cho ptree. Tích hợp phải giữ đồng thời bảng handler, bảng tên trace, user stub, prototype và các mục trong `UPROGS`; thiếu một mắt xích có thể gây lỗi liên kết hoặc lời gọi không được nhận diện.

2.2 Trace: trạng thái riêng và kế thừa

`trace(mask)` lưu mask 32 bit trong `proc.tracemask`. Với syscall số s, điều kiện chọn là bit s của mask được bật. Dispatcher kiểm tra số bit trước phép dịch và log sau khi handler trả về, theo dạng `PID: syscall NAME -> RESULT`. Giá trị trả về được in với kiểu có dấu 64 bit để giữ cả kết quả âm.

Mask được khởi tạo khi cấp slot, xóa khi thu hồi và sao chép từ cha trước khi con được chuyển sang runnable. `exec` thay không gian địa chỉ nhưng giữ đối tượng process, nên giữ mask. Thay mask ở một process không sửa mask của các process khác. `trace(0)` tắt theo dõi; nếu mask mới chứa bit trace thì chính syscall này cũng được log. `exit` không trở lại dispatcher nên không có dòng kết quả trả về.

CLI nhận mask thập phân từ 0 đến 2147483647, kiểm tra ký tự và tràn trước phép nhân/cộng. Chương trình chuyển `argv+2` trực tiếp cho `exec` để giữ các đối số của lệnh đích, trả mã 1 khi usage, mask hoặc exec lỗi. API kernel giữ toàn bộ bit của đối số `int`; test riêng bao phủ bit cao và mask -1.

2.3 Sysinfo: thống kê tài nguyên kernel

Giao diện. `sysinfo(struct sysinfo *)` trả 0 khi thành công và −1 khi bộ đệm không hợp lệ. Cấu trúc dùng hai trường `uint64`: `freemem` là số byte bộ nhớ vật lý còn trống; `nproc` là số slot có trạng thái khác `UNUSED`. Đơn vị byte được giữ thống nhất giữa kernel, chương trình test và báo cáo.

Bộ nhớ trống. Helper trong `kernel/kalloc.c` duyệt free list dưới `kmem.lock`. Mỗi phần tử tương ứng một trang, nên tổng bằng số trang nhân `PGSIZE`. Kết quả không phải tổng RAM giả lập và không tính các trang đã cấp cho kernel, page table, process hoặc pipe là bộ nhớ trống.

Số process. Helper trong `kernel/proc.c` duyệt tối đa `NPROC` slot. Mỗi slot được kiểm tra dưới `p->lock`; chỉ loại `UNUSED`. Vì vậy, process đang ngủ, đang chạy hoặc `ZOMBIE` chưa được `wait` thu hồi đều được tính. Test có đồng bộ pipe để giữ con tồn tại khi đo, và test riêng quan sát zombie trước khi cha gọi wait.

Chuyển dữ liệu về user space. Handler nhận địa chỉ qua `argaddr`; đây chỉ là thao tác lấy giá trị tham số, chưa xác nhận con trỏ hợp lệ. Điều kiện phạm vi dùng dạng `sizeof(info) <= sz - addr` sau khi kiểm tra `addr < sz`, tránh phép cộng có thể tràn. Sau khi điền cấu trúc cục bộ, kernel gọi `copyout` và truyền lỗi về nếu trang không tồn tại, là guard page hoặc không có quyền ghi trong user space.

Phạm vi đồng thời. Free list và bảng process được quan sát qua các khóa khác nhau. Không có giao dịch khóa toàn hệ thống bao quanh cả hai phép đếm; vì vậy, hai trường không được hiểu là một ảnh chụp nguyên tử cùng thời điểm. Khi test cần so sánh trước/sau chính xác, tiến trình con được giữ ở trạng thái ổn định bằng pipe và chỉ được giải phóng sau khi đo.

Độ phức tạp. Nếu free list có F trang, chi phí thống kê là $O(F + \text{NPROC})$ và bộ nhớ phụ là $O(1)$. Cách duyệt phù hợp mục đích học thuật và quy mô xv6; báo cáo chưa đo độ trễ hoặc đề xuất dùng syscall này như một bộ giám sát tần suất cao.

Kiểm chứng sysinfo

Chương trình do thành viên bàn giao kiểm tra dữ liệu cơ bản, cấp/thu hồi bộ nhớ, fork/wait và địa chỉ lỗi, kết thúc bằng `sysinfotest: OK`. Test tích hợp bổ sung đơn vị byte, guard page, text chỉ đọc, cấu trúc nằm qua hai trang, giới hạn logical heap và cách đếm zombie.

2.4 Ptree và pstree: tách thu thập khỏi hiển thị

Giao diện dữ liệu. `ptree(buf,max)` trả số bản ghi đã ghi, tối đa `max`; trả -1 khi đầu vào hoặc copyout lỗi. Mỗi bản ghi chứa PID, PPID, state, kích thước không gian địa chỉ user theo byte và tên 16 byte. Trên ABI RV64 đang dùng, cấu trúc có kích thước 40 byte, gồm cả padding để căn trường 64 bit.

Thu thập trong kernel. Giá trị `max<=0` và con trỏ lỗi bị từ chối; max lớn hơn `NPROC` được giới hạn về `NPROC`. Kernel bỏ các slot UNUSED. Khi đọc quan hệ cha, kernel lấy `wait_lock` trước `p->lock`, theo thứ tự đã dùng cho parent pointer trong xv6; thả khóa trước khi copyout. Mỗi bản ghi được zero toàn bộ trước khi điền, bao gồm padding và phần đuôi tên chưa dùng.

Giới hạn bộ đệm. Kiểm tra mapping đơn thuần chưa đủ: sau khi tăng heap một byte, phần còn lại của trang vẫn được map nhưng nằm ngoài `sz`. Vì vậy, mỗi đích ghi được kiểm tra với logical size trước khi copyout. Nếu lỗi xuất hiện ở một bản ghi về sau, syscall trả -1; những bản ghi trước đó có thể đã được ghi, nên user chỉ sử dụng kết quả khi giá trị trả về không âm.

Dựng cây ở user space. `pstree` lấy tối đa 64 bản ghi vào mảng toàn cục, tìm node không có cha trong snapshot rồi in đệ quy với hai khoảng trắng mỗi cấp. Mảng `printed` bảo đảm mỗi node chỉ được in một lần; vòng cuối in các node còn lại để xử lý snapshot thiếu parent. Kernel không in cây hoặc giữ bộ đệm trình bày.

Mỗi dòng gồm PID, tên, state dạng số và `memsize`. Trường memsize là kích thước không gian địa chỉ user của process, không phải số trang RAM còn trống hoặc RSS. Bản tích hợp bỏ phép ép về `int`; hàm in thập phân nhỏ giữ đủ `uint64`, vì `printf` user trong baseline cũng thu hẹp khi xử lý `%lu`.

Chi phí và giới hạn snapshot. Thu thập có chi phí $O(\mathrm{NPROC})$ và không cấp phát bộ nhớ kernel mới. Dựng cây quét mảng để tìm con nên có chi phí $O(n^2)$, với $n \leq 64$. Toàn bộ bảng không được đóng băng; khi process exit, exec hoặc reparent trong lúc duyệt, quan hệ giữa các bản ghi có thể thay đổi. Chương trình biểu diễn snapshot thu được, không khẳng định tính nguyên tử toàn hệ thống.

3 Kiểm thử và đánh giá

3.1 Khả năng tái lập và tiêu chí

Script `tools/verify-lab02.py` xuất commit cố định bằng `git archive` vào thư mục tạm, chạy clean, build và các bộ test, rồi lưu SHA, cấu hình công cụ, mã thoát và hash nhật ký. Nhờ đó, kết quả không phụ thuộc các object hoặc `fs.img` đã có ở checkout làm việc. C được biên dịch với `-Wall -Werror`; helper chỉ được cài vào image test, không thêm vào filesystem chuẩn.

```
python3 tools/verify-lab02.py --commit 626c6e0
```

Các suite có thể chạy riêng bằng các lệnh sau:

```
make check-trace
make check-lab02
make check-lab01
./grade-lab02-kernel
```

Bộ test làm mới image khi cần, nên dữ liệu thử tay trên image có thể bị thay thế. Kết quả được chấp nhận khi đúng dữ liệu, mã thoát và hoàn tất trong giới hạn thời gian; không coi output mẫu có PID cố định là điều kiện đúng.

3.2 Ma trận kiểm thử trên bản tích hợp

Bảng 2. Các bộ kiểm thử được chạy lại trên cùng commit.

Bộ kiểm thử	Nội dung và phạm vi	Kết quả
Trace, 7 nhóm	Mask đơn/toàn bộ/0, thay mask, fork/exec, sibling, tái sử dụng slot, CLI, forkforkfork.	7/7; 70/70.
Lab 2, 9 nhóm	Sysinfotest của thành viên; heap boundary; sysinfo; ptree; quan hệ và zombie; đồng thời; cây thật; trace hai syscall mới; in 64 bit.	9/9; 90/90.
MIT utility	Các test tương ứng sleep, pingpong, primes kế thừa từ Lab 1.	5/5.
TV1 mở rộng	Đối số, chạy lặp, pipeline, mã thoát và tick của ba tiện ích process.	5/5; 50/50.
Lab 1 tích hợp	cp, diff, tree, du; lỗi tài nguyên, định dạng 64 bit, luồng shell chung.	7/7; 70/70.
Kernel regression	`usertests -q` trên kernel tích hợp cả ba syscall.	`ALL TESTS PASSED`.

Các điểm trong bảng là thang của script nội bộ, không phải điểm chấm môn học và không cộng thành một điểm đánh giá chung. Nhật ký cuối nằm trong `docs/lab02/validation/`; bảng được đối chiếu với kết quả thực thi, không dựa trên việc đã tích ô checklist.

3.3 Minh chứng chạy thật và phân tích

Hình 2 kiểm chứng bảng tên và handler sau khi ghép nhánh. Mask `25165824` chọn bit 23 và 24; cùng một process gọi sysinfo thành công rồi nhận số bản ghi từ ptree. Không có log cho các syscall khác khi mask chỉ chọn hai bit này.

```
$ trace 25165824 lab02check snapshots
3: syscall sysinfo -> 0
3: syscall ptree -> 3
LAB02_SNAPSHOTS_OK
```

Hình 2. Output QEMU được trích từ `qemu-combined.txt`; PID có thể thay đổi khi chạy lại.

Hình 3 thể hiện các process init, shell và chính chương trình pstree theo ba mức thụt lề. Đây là cây quan sát tại thời điểm test, không phải cây minh họa được điền thủ công. Các phép kiểm tra PID/PPID nhiều thế hệ dùng pipe giữ process tồn tại; bộ kiểm tra đồng thời thêm fork/exit trong khi hai worker lặp truy vấn snapshot.

```
$ pstree
1 init state=2 mem=20480
  2 sh state=2 mem=24576
    3 pstree state=4 mem=20480
```

Hình 3. Cây tiến trình thực tế, trích từ `qemu-pstree.txt`. State và kích thước phụ thuộc lần chạy.

3.4 Lỗi được phát hiện trong rà soát tích hợp

Hai test trước sửa ghi nhận lỗi có thể tái hiện: ptree nhận buffer ở logical break chưa căn trang và trả các byte đuôi name chưa khởi tạo. Bản sửa thêm kiểm tra `sz` cho từng record và zero struct; cùng các test đó đạt sau sửa. Minh chứng trước sửa được giữ ở `before-badbuf.txt` và `before-ptree.txt`; các phép kiểm tra cuối bổ sung cả padding, guard page và text không có quyền ghi.

Một lỗi về độ rộng dữ liệu được xử lý bằng cách giữ memsize 64 bit đến lúc in. Test formatter dùng 2^{32} và $2^{64} - 1$ để phát hiện thu hẹp kiểu; các giá trị này là dữ liệu tổng hợp cho hàm trình bày, không phải bộ nhớ process thật trên máy giả lập 128 MiB.

4 Khó khăn và giới hạn

Tích hợp trên cùng file. Ba nhánh cùng thêm dispatcher, stub và prototype. Xung đột được giải quyết bằng cách giữ đủ ba syscall cùng ID đã thống nhất, không chọn toàn bộ file của một nhánh để thay phần còn lại. Nhánh ptree còn có thay đổi CRLF rộng và xóa file nguồn báo cáo Lab 1; merge chuẩn hóa LF, giữ lại báo cáo cũ và thêm quy ước line ending. Các commit gốc của thành viên vẫn là tổ tiên của bản tích hợp.

Con trỏ và byte padding. Dữ liệu có mapping chưa chắc nằm trong logical heap; một cấu trúc có tất cả trường hợp lệ vẫn có thể chứa padding chưa khởi tạo. Hai vấn đề này cần kiểm thử riêng, không chỉ kiểm tra output cây hoặc dòng `sysinfotest: OK`. Dùng `copyout` là điều kiện cần để kiểm tra page và quyền truy cập; kiểm tra phạm vi và khởi tạo buffer vẫn phải làm ở handler/helper.

Đồng thời và hiệu năng. Việc lấy từng khóa giúp bảo vệ cấu trúc đang đọc, nhưng không tạo snapshot nguyên tử giữa tất cả slot hoặc giữa bộ nhớ và process. Số PID, state, freemem và thời gian hoàn tất có thể thay đổi. Trace phát sinh console I/O và có thể làm đổi lịch thực thi. Các test đồng thời đã đạt chỉ bao phủ các workload đã chạy, không chứng minh mọi xen kẽ hoặc độ trễ tối đa.

5 Phân công và đóng góp

Bảng 3. Phần bàn giao theo phân công và lịch sử Git.

Thành viên	Phần triển khai được ghi nhận
Phan Lê Đăng Khoa	Trace, chương trình trace và self-test; nhánh `lab2-trace`, head `0d06ca9`. Theo phân công phụ trách điều phối và tích hợp.
Âu Bảo Trân	Sysinfo, helper thống kê bộ nhớ/process, sysinfotest; nhánh `lab2-sysinfo`, commit `05da92d`.
Dương Nhựt Duy	Ptree, header dữ liệu và pstree; nhánh `lab2-ptree`, head `c0a4bb7`.

Các sửa lỗi, test bổ sung và chuẩn hóa khi tích hợp được ghi trong commit `626c6e0` và `docs/lab02/REVIEW.md`. Bảng ghi nhận phạm vi bàn giao từ Git, không quy đổi số dòng hay số commit thành phần trăm đóng góp. Rà soát kỹ thuật tích hợp trong hồ sơ không thay cho biên bản review chéo mang tên từng thành viên.

6 Kết luận

Lab 2 đã bổ sung đủ ba syscall trên cùng kernel xv6-riscv. Trace minh họa trạng thái riêng của process và tính kế thừa; sysinfo nối thống kê kernel với bộ đệm user; ptree phân tách thu thập thông tin khỏi dựng cây. Bản tích hợp giữ lịch sử ba nhánh và vượt qua các bộ test chức năng cùng hồi quy được nêu trong Bảng 2.

Kết quả rà soát cho thấy kiểm tra mapping, kiểm tra logical size và khởi tạo dữ liệu trước copyout là ba trách nhiệm khác nhau. Đầu ra đúng trong trường hợp thông thường chưa đủ phát hiện lỗi padding hoặc bộ đệm nằm ngoài heap nhưng còn trong trang. Các giới hạn còn lại gồm snapshot không nguyên tử, khả năng đã ghi một phần khi copyout lỗi và tác động I/O của trace; chưa có kết quả đo hiệu năng hoặc chứng minh đầy đủ tính đúng đồng thời.

Tài liệu tham khảo

[1] Đề bài môn Hệ điều hành, *Project 2 – System call*, `Lab2_OS_System_Call.docx`. Yêu cầu trace, sysinfo, process tree và quy định bài nộp.

[2] MIT, *Lab: System calls*, 6.S081, 2020. Đối chiếu hướng dẫn trace/sysinfo ngày 04/11/2026. `https://pdos.csail.mit.edu/6.828/2020/labs/syscall.html`

[3] R. Cox, F. Kaashoek, R. Morris, *xv6: a simple, Unix-like teaching operating system*, bản RISC-V, 2024. Chương 2–4 và 6 dùng để đối chiếu tổ chức kernel, syscall, đối số và thứ tự khóa. Truy cập ngày 04/11/2026. `https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf`

[4] Nhóm thực hiện, *Mã nguồn và hồ sơ Lab 2*, commit kiểm chứng `626c6e0`; các nhánh thành viên, `docs/lab02/REVIEW.md` và chương trình kiểm thử trong `tests/`. `https://github.com/EndlessMelody/os-lab2`

[5] Nhóm thực hiện, *Minh chứng kiểm thử và báo cáo ngày 04/11/2026*, `docs/lab02/validation/` và `docs/lab02/figures/`. Lệnh, mã thoát, SHA commit và hash nhật ký lưu trong `run-context.json`; figure có hồ sơ nguồn và dòng trích trong `manifest.json`.